ean universidad

acreditada
en alta calidad
res. N° 023654 del Mineducación
10/12/21, vigencia 10/12/27

PYTHON PARA PROCESAMIENTO, CALIDAD Y AUTOMATIZACIÓN DE DATOS

Sesión 3 · Calidad de datos: perfilado, reglas y métricas

De "parece que está bien" a "está bien en un 97,2 %"

Jawy Andrés Romero Pinto · Jueves 5 de noviembre de 2026 · 08:00 – 10:00 a. m.

SOMOS LA U. DEL
EMPRENDIMIENTO
SOSTENIBLE.

Acreditaciones:

Agenda de hoy

Jueves 5 de noviembre · 08:00 – 10:00 a. m.

08:00	**Bienvenida y reto de ayer**	Los días de respuesta de las PQRS: lo que encontraste son reglas sin escribir.
08:05	**Por qué importa la calidad**	Un dato malo en un fondo de pensiones es un problema para una persona real.
08:10	**Las 7 dimensiones**	Completitud, unicidad, validez, consistencia, exactitud, oportunidad, integridad.
08:15	**Perfilado**	La radiografía de una tabla en una función.
08:20	**Reglas de validación**	Máscaras booleanas, expresiones regulares, reglas de negocio y cruces.
08:32	**Catálogo, motor y reporte**	Las reglas como datos; una función que las aplica todas; semáforo e índice.
08:42	**Laboratorio**	Construyes el reporte de calidad de afiliados, PQRS y aportes (70 min).
09:55	**Cierre y reto**	Lo que te llevas y el puente a la automatización de mañana.

01

De dónde venimos

Ver el problema, corregirlo y, hoy, medirlo.

Ver, corregir, medir

Cada sesión responde una pregunta distinta sobre el mismo archivo

Sesión	Pregunta	Qué hicimos	Herramienta
1 · Fundamentos	¿Qué hay en el archivo?	Vimos 39 ciudades, salarios como texto, 90 duplicados	info, value_counts, filtros
2 · Limpieza	¿Cómo lo corrijo?	strip, map, to_numeric, to_datetime, drop_duplicates, merge	pandas sobre una copia
3 · Calidad (hoy)	¿Quedó bien? ¿Puedo confiar?	Reglas medibles que se ejecutan siempre igual	máscaras, catálogo, motor
4 · Automatización	¿Y el archivo del mes que viene?	El motor corre solo con cada archivo nuevo	scripts, funciones, logs

Limpiar sin medir es fe. Medir sin limpiar es diagnóstico. Lo profesional: **medir, limpiar y volver a medir**.

Por qué importa la calidad

En un fondo de pensiones, un dato malo es un problema para una persona real

Si el dato está mal

- Un aporte mal registrado son **semanas faltantes** en la historia laboral.
- Una fecha de nacimiento errada cambia la **edad de pensión**.
- Un documento duplicado parte la historia de una persona en dos.
- Un correo inválido son notificaciones que **nunca llegan**.

La regla 1 - 10 - 100

- Prevenir el error en la captura cuesta **1**.
- Corregirlo después en la base cuesta **10**.
- Si llega a una decisión (una pensión, un reporte a un ente de control) cuesta **100**.
- Medir temprano es la forma más barata de ahorrar.

Calidad no es "perfecto": es **adecuado para el uso**. Un correo errado es grave para notificar e irrelevante para un promedio de salarios.

Las 7 dimensiones de calidad

Cada una responde una pregunta distinta, todas con ejemplos de nuestros datos

Dimensión	Pregunta	Ejemplo en el caso	Cómo se mide
Completitud	¿Está el dato?	Género vacío en 64 afiliados	`notna()`
Unicidad	¿Está una sola vez?	90 filas duplicadas exactas	`~duplicated()`
Validez	¿Cumple el formato o el dominio?	Documento "593.025.079", correo sin @	`str.match`, `isin`
Consistencia	¿Concuerda con otros datos?	Ciudad Cali con departamento Bogotá D.C.	catálogo + `map`
Exactitud	¿Es plausible, refleja la realidad?	Salario negativo, nacido en 1890	`between`, `>=`
Oportunidad	¿Llega o se atiende a tiempo?	PQRS respondida después de 15 días hábiles	`busday_count`
Integridad referencial	¿Existe en la tabla maestra?	Aporte de un afiliado que no existe	`isin(maestra)`

Las siete se miden con lo mismo: una **pregunta de sí o no por registro**.

ean universidad



02

Perfilado

Antes de escribir reglas, mira qué hay en cada columna.

Perfilar: la radiografía de una tabla

Tipo, vacíos, valores distintos y un ejemplo por columna

Qué es

Un resumen **por columna**: qué tipo tiene, qué porcentaje está vacío, cuántos valores distintos hay y cómo se ve uno. Es `info()` más `nunique()` en una sola tabla.

Para qué sirve

Encontrar las señales de alerta antes de escribir reglas: un salario de tipo texto, un género con 10 valores, un id con menos distintos que filas.

Analogía con Excel

Es abrir el archivo y poner un filtro en cada columna para ver qué valores aparecen. Aquí lo hace una función, igual para cualquier archivo.

Cómo manejarlo

Escribe `perfilar(df)` una vez y úsala con afiliados, PQRS y aportes. Compara `distintos` con lo que **debería** haber: 2 géneros, 3 estados, 15 ciudades.

Abstracción desde el código

```
def perfilar(df):
  return pd.DataFrame({
    "tipo": df.dtypes.astype(str),
    "pct_nulos": df.isna().mean() * 100,
    "distintos": df.nunique(),
  })

perfilar(afiliados)
```

Una función que recibe cualquier tabla y devuelve otra tabla. El perfil no se hace a ojo: se calcula, y mañana se calcula igual con el archivo nuevo.

Perfilado de afiliados

Una fila por columna: tipo, % de vacíos, distintos y ejemplo

[1]
```python
def perfilar(df):
    """Una fila por columna: tipo, % de vacíos, distintos y un ejemplo."""
    return pd.DataFrame({
        "tipo": df.dtypes.astype(str),
        "pct_nulos": (df.isna().mean() * 100).round(2),
        "distintos": df.nunique(),
        "ejemplo": df.apply(lambda c: c.dropna().iloc[0]),
    })

print("Filas:", len(afiliados))
perfilar(afiliados).head(11)
```

```
Filas: 3120
```

	tipo	pct_nulos	distintos	ejemplo
id_afiliado	str	0.00	3030	AF100026
tipo_documento	str	0.00	2	CC
numero_documento	str	0.48	2985	940974240
nombres	str	0.00	844	PATRICIA
apellidos	str	0.00	1012	Ortiz Ortiz
fecha_nacimiento	str	0.00	2789	1971-09-13
genero	str	2.05	10	F
ciudad	str	0.00	39	Medellín
departamento	str	0.00	15	Antioquia
fecha_afiliacion	str	0.00	2585	2007-03-17
salario_base	str	1.03	1981	1819000

Qué observar

- 3.120 filas pero **3.030** id distintos: hay 90 repetidos.
- `genero` tiene **10** valores distintos (deberían ser 2); `ciudad` 39 (deberían ser 15).
- `salario_base` es tipo texto: hay números con puntos.

Cada señal de alerta del perfil es una **regla por escribir**. El perfil te dice dónde mirar; la regla te dice cuánto falla.

ean universidad

03

Reglas de validación

Una regla es una pregunta de sí o no por registro.

Una regla es una máscara booleana

True cumple, False falla; el promedio es el % de cumplimiento

Qué es

Una columna de verdadero o falso, una por registro, que responde una pregunta: ¿este estado está en la lista permitida? Es la misma máscara que usaste para filtrar en la Sesión 1.

Para qué sirve

Convertir "se ve mal" en un número: cuántos fallan y qué porcentaje cumple. Y ver exactamente cuáles fallan con `df[~ok]`.

Analogía con Excel

Una columna auxiliar con =SI(O(C2="Activo";...);1;0) y un PROMEDIO() al final. Aquí no hay columna auxiliar ni arrastre: la máscara es la columna entera.

Cómo manejarlo

Nombra cada máscara `ok_algo`. Cuenta las fallas con `(~ok).sum()`, mide con `ok.mean()`, y mira `df[~ok]` antes de decidir si es error de forma o de fondo.

Abstracción desde el código

```
ok = df["estado"].isin(
    ["Activo", "Inactivo", "Pensionado"])

(~ok).sum()        # cuántos fallan
ok.mean() * 100    # % cumplimiento
df[~ok]            # cuáles
```

True vale 1 y False vale 0, así que el promedio de la máscara **es** el porcentaje de cumplimiento. Un indicador de calidad son tres caracteres: `.mean()`.

La regla en acción: dominio de estado

isin contra la lista oficial

[2]
```python
# Validez de dominio: el estado debe ser uno de los valores permitidos
ok_estado = afiliados["estado"].isin(["Activo", "Inactivo", "Pensionado"])

print(ok_estado.head(3).tolist(), "...")           # True = cumple, False = falla
print("Fallas:", (~ok_estado).sum())
print("Cumplimiento:", round(ok_estado.mean() * 100, 2), "%")

afiliados.loc[~ok_estado, "estado"].value_counts()   # qué está fallando
```

```
[True, True, True] ...
Fallas: 94
Cumplimiento: 96.99 %
estado
inactivo      20
activo        20
Activo        19
ACTIVO        18
Pensionada    17
Name: count, dtype: int64
```

Qué observar

- La máscara es una lista de True y False, una por fila.
- 94 fallas, **96,99 %** de cumplimiento, en dos líneas.
- `df.loc[~ok]` muestra **qué** falla: minúsculas, espacios, "Pensionada".

Estas 94 fallas son de **forma** y la Sesión 2 las corrige. La regla sirve igual para medir antes y después de limpiar.

Validez con expresiones regulares

Describir un formato en vez de enumerar casos

Qué es

Un patrón que describe cómo debe verse un texto: `^\d{6,10}$` es "solo dígitos, entre 6 y 10, de principio a fin". `str.match` lo prueba en toda la columna.

Para qué sirve

Validar documentos, celulares y correos sin escribir cien `if`. Un patrón de una línea reemplaza una lista infinita de casos.

Analogía con Excel

Es la validación de datos de Excel ("solo números de 10 dígitos") pero aplicada **después**, sobre lo que ya llegó, y sin que nadie la pueda saltar.

Cómo manejarlo

Aprende cinco símbolos: `^` inicio, `$` fin, `\d` dígito, `{n,m}` repeticiones, `\w` letra o dígito. Usa `na=False` para que el vacío cuente como falla. Prueba el patrón en regex101.com.

Abstracción desde el código

```
ok_doc = df["numero_documento"]
    .str.match(r"^\d{6,10}$", na=False)

ok_tel = df["telefono"]
    .str.match(r"^3\d{9}$", na=False)
```

El formato correcto queda escrito como un patrón, no como una explicación en un correo. La misma línea valida el archivo de hoy y el de dentro de un año.

Documento, celular y correo

Tres patrones, tres columnas, una línea cada uno

[3]
```
# ^ inicio  $ fin  \d dígito  {6,10} entre 6 y 10 veces
ok_doc = afiliados["numero_documento"].str.match(r"^\d{6,10}$", na=False)
ok_tel = afiliados["telefono"].str.match(r"^3\d{9}$", na=False)
ok_email = afiliados["email"].str.match(r"^[\w.+-]+@[\w-]+\.[\w.]+$",
na=False)

print("Fallas documento:", (~ok_doc).sum(), "| teléfono:", (~ok_tel).sum(),
      "| email:", (~ok_email).sum())
print(afiliados.loc[~ok_doc, "numero_documento"].dropna().head(4).tolist())
afiliados.loc[~ok_email, "email"].dropna().head(4).tolist()
```

```
Fallas documento: 166 | teléfono: 94 | email: 75
['593.025.079', '302.198.956', ' 1005862867 ', '499.903.434']
['sin correo', 'natalia.suarez592correo.com', 'sin correo', 'sin correo']
```

Qué observar

- Documento: **166** fallas; celular: 94; correo: 75.
- Fallan los puntos de miles, los espacios y los vacíos (`na=False`).
- El correo sin @ y el texto "sin correo" son fallas distintas: forma y contenido.

"593.025.079" es un problema de **forma** (se limpia). "sin correo" es un dato que **no existe** y hay que pedirlo. La regla encuentra ambos; la decisión es tuya.

Reglas de negocio: exactitud

Conocimiento del dominio hecho código

Qué es

Una regla que no sale del formato sino de cómo funciona el negocio: nadie se afilia antes de los 18, ningún IBC es menor al salario mínimo, nadie nace en 1890.

Para qué sirve

Encontrar datos que **tienen el formato correcto y aun así son falsos**. Un salario de -1.500.000 es un número perfecto y un dato imposible.

Analogía con Excel

Es el formato condicional que pinta de rojo la celda con fecha futura. Aquí el rojo se cuenta y se reporta.

Cómo manejarlo

Primero convierte al tipo correcto (fecha, número). Luego escribe el rango con `between` o la comparación con la constante del negocio (`SMMLV`). Separa las fallas por causa.

Abstracción desde el código

```
edad = (HOY - nac).dt.days // 365.25
ok_edad = edad.between(18, 100)

salario = pd.to_numeric(
    df["salario_base"], errors="coerce")
ok_salario = salario >= SMMLV
```

Las constantes del negocio (`SMMLV`, 18, 100) viven en un solo lugar con nombre. Cuando cambie el salario mínimo, cambia una línea y todas las reglas se actualizan.

Edad y salario mínimo

Formato correcto no significa dato correcto

[4]
```
def a_fecha(serie):                          # acepta 1971-09-13 y 13/09/1971
    iso = pd.to_datetime(serie, format="%Y-%m-%d", errors="coerce")
    dmy = pd.to_datetime(serie, format="%d/%m/%Y", errors="coerce")
    return iso.fillna(dmy)

edad = (HOY - a_fecha(afiliados["fecha_nacimiento"])).dt.days // 365.25
ok_edad = edad.between(18, 100)
salario = pd.to_numeric(afiliados["salario_base"], errors="coerce")
ok_salario = salario >= SMMLV

print("Edades fuera de 18-100:", (~ok_edad).sum(), "->", sorted(int(e) for e
in edad[~ok_edad].unique()))
print("Salario bajo el mínimo:", (~ok_salario).sum(),
      "= no numérico", salario.isna().sum(), "+ negativo", (salario <
0).sum(),
      "+ entre 0 y SMMLV", ((salario >= 0) & (salario < SMMLV)).sum())
```

```
Edades fuera de 18-100: 13 -> [-5, 126, 136]
Salario bajo el mínimo: 174 = no numérico 95 + negativo 15 + entre 0 y SMMLV 64
```

Qué observar

- `a_fecha` acepta los dos formatos de fecha de la Sesión 2.
- 13 edades imposibles: **-5, 126 y 136** años.
- 174 fallas de salario, separadas por causa: 95 no numéricos, 15 negativos, 64 bajo el mínimo.

Separar las fallas por causa decide el siguiente paso: los no numéricos se **limpian**; los negativos y los bajos del mínimo se **devuelven** al área.

Consistencia: entre columnas y entre tablas

El dato está bien, pero no concuerda con otro

Qué es

Una regla que compara dos datos que deben coincidir: la ciudad con su departamento según el catálogo, o el estado "pensionado" con la edad mínima para serlo.

Para qué sirve

Detectar errores que ninguna columna muestra sola. Cali es válida, Bogotá D.C. es válido, "Cali, Bogotá D.C." es imposible.

Analogía con Excel

Un BUSCARV del departamento esperado y un =SI(esperado=real). El catálogo de la Sesión 2 vuelve a servir.

Cómo manejarlo

Usa `map` con un diccionario del catálogo para obtener el valor esperado. Aplica el patrón **"no aplica = cumple"**: la regla del pensionado solo evalúa a los pensionados.

Abstracción desde el código

```
esperado = df["ciudad"].map(mapa)
ok_depto = (esperado.isna()
    | (esperado == df["departamento"]))

ok_pension = (~es_pensionado
    | (edad >= edad_minima))
```

El `|` con la negación es el patrón de toda regla condicional: o no te aplica, o la cumples. Así los 2.500 no pensionados no "fallan" una regla que no es para ellos.

Ciudad con departamento; pensionado con edad

Dos reglas de consistencia con el patrón no aplica = cumple

[5]
```python
# ¿El departamento corresponde a la ciudad, según el catálogo oficial?
mapa = dict(zip(cat_ciudades["ciudad"], cat_ciudades["departamento"]))
esperado = afiliados["ciudad"].map(mapa)
ok_depto = esperado.isna() | (esperado == afiliados["departamento"])
print("Ciudad-departamento incoherentes:", (~ok_depto).sum())

# ¿Un pensionado tiene la edad mínima? (57 mujeres / 62 hombres)
g = afiliados["genero"].str.strip().str.upper().str[0]
es_pens =
afiliados["estado"].str.strip().str.lower().str.startswith("pensionad")
edad_min = pd.Series(np.where(g == "F", 57, 62), index=afiliados.index)
ok_pension = ~es_pens | (edad >= edad_min)
print("Pensionados sin la edad mínima:", (~ok_pension).sum())

afiliados.loc[~ok_depto, ["ciudad",
"departamento"]].assign(esperado=esperado).head(3)
```

```
Ciudad-departamento incoherentes: 26
Pensionados sin la edad mínima: 48
```

	ciudad	departamento	esperado
22	Bucaramanga	Caldas	Santander
58	Barranquilla	Valle del Cauca	Atlántico
76	Cali	Bogotá D.C.	Valle del Cauca

Qué observar

- 26 afiliados con ciudad y departamento que **no concuerdan** según el catálogo.
- 48 pensionados sin la edad mínima (57 mujeres, 62 hombres).
- `np.where(g == "F", 57, 62)`: la edad mínima depende de otra columna.

Bucaramanga en Caldas no es un error de escritura: alguien capturó mal uno de los dos. La regla no sabe cuál; lo marca para que una persona decida.

Oportunidad e integridad referencial

Otras tablas, las mismas ideas

[6]

```
# PQRS, oportunidad: ¿se respondió en máximo 15 días hábiles?
rad = pd.to_datetime(pqrs["fecha_radicacion"])
resp = pd.to_datetime(pqrs["fecha_respuesta"])
m = resp.notna()
dias = pd.Series(np.nan, index=pqrs.index)
dias[m] = np.busday_count(rad[m].values.astype("datetime64[D]"),
                          resp[m].values.astype("datetime64[D]"))
ok_oportuna = dias.isna() | (dias <= 15)
print("PQRS tardías:", (~ok_oportuna).sum(), "| mediana:", dias.median(),
"días | máx:", dias.max())

# Aportes, integridad referencial: ¿el afiliado existe en la tabla maestra?
ok_ref = aportes["id_afiliado"].isin(afiliados["id_afiliado"])
print("Aportes huérfanos:", (~ok_ref).sum(), "->", aportes.loc[~ok_ref,
"id_afiliado"].head(3).tolist())
```

```
PQRS tardías: 165 | mediana: 7.0 días | máx: 39.0
Aportes huérfanos: 8 -> ['AF800320', 'AF809588', 'AF808642']
```

Qué observar

- `np.busday_count` cuenta **días hábiles** entre dos fechas.
- 165 PQRS respondidas después de 15 días hábiles; la mediana es 7 y el máximo 39.
- 8 aportes de marzo son de afiliados que **no existen** en la tabla maestra.

Un aporte huérfano es dinero sin dueño. La regla de integridad es un `isin` contra la tabla maestra: la misma herramienta de la Sesión 1, con otra pregunta.

ean universidad

04

Catálogo, motor y reporte

Las reglas como datos, una función que las aplica todas y un número para gerencia.

El catálogo: las reglas como datos

Una lista de diccionarios, una entrada por regla

Qué es

Una lista donde cada regla es un diccionario con nombre, dimensión, severidad y la función que la evalúa. Las reglas dejan de estar regadas en celdas sueltas.

Para qué sirve

Agregar, quitar o cambiar una regla sin tocar el programa que las ejecuta. Documentar qué se valida y con qué gravedad, en un solo lugar legible.

Analogía con Excel

La hoja "Reglas" del documento de calidad del área: nombre, dimensión, crítica o advertencia. Aquí la hoja además **se ejecuta**.

Cómo manejarlo

Cada regla es una `lambda df:` que devuelve una máscara. Severidad crítica para lo que manda a cuarentena (sin documento, duplicado); advertencia para lo que se marca y sigue (correo mal escrito).

Abstracción desde el código

```
REGLAS = [
  {"nombre": "documento_unico",
   "dimension": "Unicidad",
   "severidad": "crítica",
   "valida": lambda df:

~df.duplicated("numero_documento")},
  # una entrada por regla
]
```

La regla es un dato y la función que la aplica es otra cosa. Separar las dos es lo que permite que mañana el catálogo crezca sin reescribir nada: es el principio de todo motor de reglas.

El catálogo de afiliados

12 reglas en el deck, 15 en el laboratorio

[7]
```python
def regla(nombre, dimension, severidad, valida):
    return {"nombre": nombre, "dimension": dimension, "severidad": severidad, "valida": valida}

PATRON_EMAIL = r"^[\w.+-]+@[\w-]+\.[\w.]+$"
REGLAS = [
  regla("documento_presente", "Completitud", "crítica", lambda d: d["numero_documento"].notna()),
  regla("genero_presente", "Completitud", "advertencia", lambda d: d["genero"].notna()),
  regla("fila_no_duplicada", "Unicidad", "crítica", lambda d: ~d.duplicated()),
  regla("documento_unico", "Unicidad", "crítica", lambda d: ~d.duplicated(subset=["numero_documento"])),
  regla("documento_formato", "Validez", "crítica", lambda d: d["numero_documento"].str.match(r"^\d{6,10}$",
na=False)),
  regla("email_formato", "Validez", "advertencia", lambda d: d["email"].str.match(PATRON_EMAIL, na=False)),
  regla("genero_dominio", "Validez", "advertencia", lambda d: d["genero"].isin(["F", "M"])),
  regla("estado_dominio", "Validez", "advertencia", lambda d: d["estado"].isin(["Activo", "Inactivo",
"Pensionado"])),
  regla("ciudad_en_catalogo", "Validez", "advertencia", lambda d: d["ciudad"].isin(cat_ciudades["ciudad"])),
  regla("edad_18_100", "Exactitud", "crítica", lambda d: ((HOY - a_fecha(d["fecha_nacimiento"])).dt.days //
365.25).between(18, 100)),
  regla("salario_minimo", "Exactitud", "crítica", lambda d: pd.to_numeric(d["salario_base"], errors="coerce") >=
SMMLV),
  regla("depto_coherente", "Consistencia", "advertencia", lambda d: d["ciudad"].map(mapa).isna() |
(d["ciudad"].map(mapa) == d["departamento"])),
]
print(len(REGLAS), "reglas. Una regla es un dato:", REGLAS[4]["nombre"], "|", REGLAS[4]["dimension"], "|",
REGLAS[4]["severidad"])
```

```
12 reglas. Una regla es un dato: documento_formato | Validez | crítica
```

Qué observar

- `regla(...)` construye el diccionario; cada línea es una regla completa.
- Nombre, dimensión y severidad son **texto**: se pueden filtrar, contar y reportar.
- `valida` es una función que recibe la tabla y devuelve la máscara.

Lee el catálogo como un documento: cualquiera del área puede revisar si las reglas son las correctas sin saber programar. Lo que no puede revisar es una regla escondida en una celda.

El motor: una función aplica todas las reglas

Recibe una tabla y un catálogo, devuelve el reporte

Qué es

Un `for` sobre la lista de reglas: ejecuta cada `valida`, cuenta las fallas, calcula el porcentaje y anota en cada registro qué reglas falló.

Para qué sirve

Un reporte de calidad **repetible**: la misma función para afiliados, PQRS y aportes, hoy y cada mes. Y una columna `errores` por registro para trazabilidad.

Analogía con Excel

Es la macro que recorre la hoja "Reglas" y llena la hoja "Resultados". Solo que aquí son 15 líneas y no dependen de que el archivo tenga las celdas en el mismo sitio.

Cómo manejarlo

Devuelve tres cosas: el reporte (una fila por regla), los errores por registro y la marca de falla crítica. `fillna(False)`: una máscara con vacíos trataría el vacío como falla, que es lo correcto.

Abstracción desde el código

```
def evaluar(df, reglas):
  for r in reglas:
    ok = r["valida"](df)
    filas.append({"regla": r["nombre"],
      "fallas": (~ok).sum(),
      "cumplimiento_%": ok.mean() * 100})
  return pd.DataFrame(filas), errores,
critica
```

Separar el **qué** (el catálogo) del **cómo** (el motor) es la idea más importante del día. El motor se escribe una vez; los catálogos son tantos como tablas tengas.

El reporte de calidad

Una fila por regla, ordenada de peor a mejor

[8]
```python
def evaluar(df, reglas):
    filas = []
    errores = pd.Series("", index=df.index)
    critica = pd.Series(False, index=df.index)
    for r in reglas:
        ok = pd.Series(r["valida"](df),
index=df.index).fillna(False).astype(bool)
        filas.append({"regla": r["nombre"], "dimension": r["dimension"],
                      "severidad": r["severidad"], "fallas":
int((~ok).sum()),
                      "cumplimiento_%": round(ok.mean() * 100, 2)})
        errores = errores.where(ok, errores + r["nombre"] + "; ")
        if r["severidad"] == "crítica":
            critica = critica | ~ok
    return pd.DataFrame(filas), errores.str.rstrip("; "), critica

def semaforo(p):
    return "VERDE" if p >= 98 else ("AMARILLO" if p >= 95 else "ROJO")

reporte, errores, critica = evaluar(afiliados, REGLAS)
reporte["semaforo"] = reporte["cumplimiento_%"].map(semaforo)
reporte.sort_values("cumplimiento_%").head(7)
```

	regla	dimension	severidad	fallas	cumplimiento_%	semaforo
8	ciudad_en_catalogo	Validez	advertencia	285	90.87	ROJO
6	genero_dominio	Validez	advertencia	246	92.12	ROJO
10	salario_minimo	Exactitud	crítica	174	94.42	ROJO
4	documento_formato	Validez	crítica	166	94.68	ROJO
3	documento_unico	Unicidad	crítica	134	95.71	AMARILLO
7	estado_dominio	Validez	advertencia	94	96.99	AMARILLO
2	fila_no_duplicada	Unicidad	crítica	90	97.12	AMARILLO

Qué observar

- Peor regla: `ciudad_en_catalogo`, **90,87 %** (285 fallas).
- `semaforo` traduce el porcentaje a verde, amarillo o rojo: 98 y 95 son los umbrales.
- Las advertencias (género, ciudad) fallan más que las críticas, pero no mandan a cuarentena.

Este reporte es lo que el analista necesita: qué corregir primero. Gerencia necesita otra cosa: **un número**. Eso sigue.

Métricas: del detalle a un número para gerencia

Reporte por regla, índice por dimensión, índice global

Nivel	Cómo se calcula	Para quién	Qué decide
Por regla	% de registros que cumplen	Analista	Qué corregir primero
Por dimensión	Promedio de las reglas de esa dimensión	Líder de datos	Dónde está el problema
Índice global	Promedio de las dimensiones	Gerencia	¿Confío o no?
Semáforo	Verde 98 % o más · Amarillo entre 95 y 98 · Rojo menos de 95	Todos	Lectura en dos segundos

Severidad decide el destino del registro: una falla **crítica** (sin documento, duplicado, salario imposible) lo manda a cuarentena; una **advertencia** (correo mal escrito) se marca y sigue.

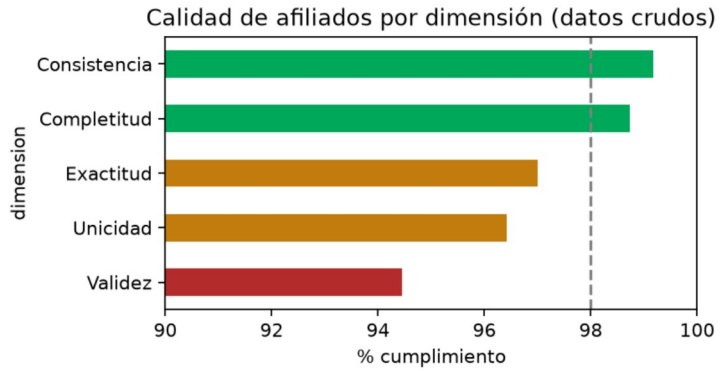
Índice por dimensión e índice global

groupby sobre el reporte y una gráfica de barras

[9]
```
indice_dim = reporte.groupby("dimension")
["cumplimiento_%"].mean().round(2).sort_values()
indice_global = round(indice_dim.mean(), 2)
print(indice_dim.to_dict())
print(f"ÍNDICE GLOBAL DE CALIDAD: {indice_global} %
({semaforo(indice_global)})")

colores = ["#00A85A" if v >= 98 else "#C27C0E" if v >= 95 else "#B42B2B" for
v in indice_dim]
ax = indice_dim.plot.barh(color=colores, xlim=(90, 100), xlabel="%
cumplimiento",
                          title="Calidad de afiliados por dimensión (datos
crudos)")
ax.axvline(98, ls="--", color="gray")
ax
```

```
{'Validez': 94.45, 'Unicidad': 96.42, 'Exactitud': 97.0, 'Completitud': 98.74,
'Consistencia': 99.17}
ÍNDICE GLOBAL DE CALIDAD: 97.16 %  (AMARILLO)
```



Qué observar

- Validez **94,45 %** es la dimensión más débil; Consistencia 99,17 % la mejor.
- Índice global **97,16 %**, amarillo.
- La gráfica es `plot.barh` con los colores del semáforo y la línea del 98.

97 % suena excelente. Pero un promedio **esconde**: la cuarentena, en la siguiente diapositiva, muestra que uno de cada siete registros tiene un error crítico.

Cuarentena: separar, no borrar

Los registros con falla crítica salen del flujo con su explicación

Qué es

Dividir la tabla en dos: **válidos** (sin fallas críticas, quizá con advertencias) y **cuarentena** (al menos una falla crítica), cada registro con la lista de reglas que falló.

Para qué sirve

Que los reportes y cruces usen solo datos confiables, y que lo defectuoso vuelva al área dueña del dato con una explicación por registro, no con "revise esto".

Analogía con Excel

Es mover las filas rojas a otra hoja llamada "Pendientes" con una columna "motivo". Aquí la columna `errores` se escribe sola.

Cómo manejarlo

Usa la marca `critica` del motor: `df[~critica]` y `df[critica]`. Cuenta `errores.value_counts()` para ver qué combinación de fallas es la más común y atacarla primero.

Abstracción desde el código

```
marcado = df.assign(errores=errores)
validos = marcado[~critica]
cuarentena = marcado[critica]


cuarentena["errores"].value_counts()
```

Borrar un registro malo lo hace desaparecer del problema y de la solución. La cuarentena lo hace visible, contable y corregible en la fuente, que es donde el error nació.

Válidos y cuarentena

El índice decía 97 %; la cuarentena dice uno de cada siete

[10]

```
marcado = afiliados.assign(errores=errores)
validos = marcado[~critica]
cuarentena = marcado[critica]
print("Válidos:", len(validos), "| Cuarentena:", len(cuarentena),
      f"({len(cuarentena) / len(marcado):.1%})")

cuarentena["errores"].value_counts().head(5)
```

```
Válidos: 2671 | Cuarentena: 449 (14.4%)
errores
salario_minimo                          112
documento_formato                       106
fila_no_duplicada; documento_unico       57
documento_unico                          24
ciudad_en_catalogo; salario_minimo       12
Name: count, dtype: int64
```

Qué observar

- 2.671 válidos y **449 en cuarentena** (14,4 %).
- La combinación más común: `salario_minimo` (112), luego `documento_formato` (106).
- 57 registros fallan dos reglas a la vez: duplicado exacto y documento repetido.

La columna `errores` es la **trazabilidad**: cada registro en cuarentena sabe por qué está ahí. Es lo que se devuelve al área, no una lista de ids.

Medir, limpiar, volver a medir

La limpieza de la Sesión 2 evaluada con las reglas de hoy

[11]
```python
def limpiar_minimo(df):                    # lo de la Sesión 2, en corto
    df = df.drop_duplicates().copy()
    df["numero_documento"] = df["numero_documento"].str.replace(r"\D", "", regex=True)
    df["genero"] = df["genero"].str.strip().str.upper().str[0]
    df["estado"] = df["estado"].str.strip().str.capitalize().replace({"Pensionada": "Pensionado"})
    df["email"] = df["email"].str.strip().str.lower()
    df["salario_base"] = pd.to_numeric(df["salario_base"].str.replace(".", "", regex=False),
errors="coerce")
    return df

limpio = limpiar_minimo(afiliados)
rep2, err2, crit2 = evaluar(limpio, REGLAS)
comparativo = pd.DataFrame({"antes": reporte.groupby("dimension")["cumplimiento_%"].mean(),
                            "despues": rep2.groupby("dimension")
["cumplimiento_%"].mean()}).round(2)
comparativo.loc["ÍNDICE GLOBAL"] = comparativo.mean().round(2)
comparativo["mejora"] = (comparativo["despues"] - comparativo["antes"]).round(2)
print("Filas:", len(afiliados), "->", len(limpio), "| Cuarentena:", int(critica.sum()), "->",
int(crit2.sum()))
comparativo
```

```
Filas: 3120 -> 3030 | Cuarentena: 449 -> 160
```

dimension	antes	despues	mejora
Completitud	98.74	98.74	0.00
Consistencia	99.17	99.21	0.04
Exactitud	97.00	98.05	1.05
Unicidad	96.42	99.28	2.86
Validez	94.45	97.23	2.78
ÍNDICE GLOBAL	97.16	98.50	1.34

Qué observar

- Índice global: **97,16 %** antes, **98,50 %** después. Cuarentena: 449 a 160.
- Mejoran Validez y Unicidad: eran problemas de **forma**.
- Completitud no cambia: la limpieza no inventa un género ni una fecha.

La limpieza arregla la forma, no la verdad. Lo que queda en cuarentena después de limpiar es lo que de verdad hay que **preguntar**.

Siguiente paso: herramientas profesionales

Lo que hoy hacemos a mano, estas librerías lo industrializan

Herramienta	Qué hace	Cuándo considerarla
ydata-profiling	Reporte HTML de perfilado automático	Primer vistazo a una tabla nueva
pandera	Esquemas y reglas sobre DataFrames de pandas	Validar dentro de scripts y pipelines
Great Expectations	"Expectativas", documentación e historial	Programas de calidad a nivel de organización
Soda, dbt tests	Reglas sobre bases de datos y data warehouses	Cuando los datos viven en SQL

Todas hacen lo que hiciste hoy: **una regla es una pregunta de sí o no por registro** y el cumplimiento es su promedio.

ean universidad

LAB

70 min

Laboratorio · Sesión 3

Abre **PY-EAN_S03_Lab_alumno.ipynb** desde GitHub → **Open in Colab** → **Copiar en Drive**.

Perfilas, escribes 10 reglas, construyes el catálogo y el motor, y produces el reporte de calidad de afiliados, PQRS y aportes con semáforo, índice por dimensión y cuarentena.

Checkpoints: Parte 2 a las 09:20 y Parte 3 (**tu reporte en pantalla**) a las 09:40.

Lo que te llevas hoy

Y el reto para mañana

Calidad es adecuado al uso

Siete dimensiones, cada una con una pregunta y una forma de medirla.

Perfilar primero

Tipo, % de vacíos, distintos y ejemplo por columna, con una función.

Una regla, una máscara

True cumple, False falla; el promedio es el % de cumplimiento.

Catálogo y motor

Las reglas son datos; una función las aplica todas y arma el reporte.

Cuarentena, no borrar

Lo defectuoso se separa con su motivo y vuelve a la fuente.

Reto para mañana

Agrega a tu catálogo **dos reglas propias** (por ejemplo, la fecha de pago del aporte dentro del mes siguiente al periodo) y di a qué dimensión pertenece cada una.

Mañana (Sesión 4 · Automatización) convertimos el catálogo y el motor en un proceso que corre solo con los nueve archivos de aportes, y uno de ellos trae una trampa.

ean universidad

acreditada
en alta calidad
res. Nº 023654 del Mineducación
10/12/21, vigencia 10/12/27



www.universidadean.edu.co

Universidad Ean: SNIES 2812 Personería Jurídica Res. 2898 del Minjusticia - 16/05/69 - Vigilada Mineducación.

Ean Legacy: Carrera 11 No. 78-47 - Fundadores: Calle 79 # 11 - 45 - Av. Chile: Calle 71 # 9 - 84 Bogotá D.C., Cundinamarca, Colombia, Suramérica.

Línea gratuita nacional: +57-01-8000-931000
Centro de contacto en Bogotá: +(57-601) 593 6161 - +(57-601) 593 6464
E-mail: informacion@universidadean.edu.co

© 2024 Todos los derechos reservados - Universidad Ean